Assignment 1

COMP3361 Natural Language Processing [Section 2A, 2025]

Released: February 6, 2025 Due: February 20, 2025 (11:59 PM)

Zoie comes from Beijing and she is learning English. She finds it sometimes difficult to decide which preposition she should use in some cases. For example, should she say "they live in a small town" or "they live at a small town"? One day she visited HKU and came across a student who happens to study COMP3361 this semester.

> "You must be an expert in language!" said Zoie.
> "We learned language model recently!" said the student with excitement.
> "That's nice! Could you please write a program for me that tells me what preposition I should use in a sentence," asked Zoie with her eyes shining like stars.
> "Sure! Anything!" said the student.

And here comes the **assignment 1**.

Reading Materials: We recommend that you read J&M Ch. 3 and M. Collins, Notes 1 before doing this assignment.

Resources: You can access all related resources at `https://github.com/XinyuanWangCS/COMP3361-2026/tree/main/A1`.

Code: We provide a Jupyter Notebook template. You can use it on your local environment or Google Colab (recommended) to finish it. The programming language is Python, building on the provided starter code. Some useful packages you may need: `nltk`, `numpy`.

Generative AI Policy: If you use any AI tools, you **must** include a dedicated section titled "Declaration of AI Use" at the end of your notebook. In this section, you must explicitly detail **how** these tools were used for **each specific part** of the assignment (e.g., "Used ChatGPT to debug the plotting code in Section 2.1", "Used Copilot to generate boilerplate class structure in Section 2.2"). Failure to include a complete and accurate "Declaration of AI Use" will be treated as a violation of the HKU Generative AI and Academic Integrity Policy and may be subject to investigation and disciplinary action in accordance with university regulations

Grade: We will evaluate your grade based on the code quality, output results/log, and discussion.

Submit: You should submit two files: `UniversityNumber.ipynb` file and final prediction file `University-Number.test.out1`, `University-Number.test.out2` of Section 2 to Moodle. All code and discussion should be included in your submitted `.ipynb` file. Make sure your code does not use your local files so that the results are reproducible. Before submitting, please run your notebook and keep all running logs so that we can check the results.

1 N-gram Language Model (30%)

A language model is a probability function p that assigns probabilities to word sequences such as $\vec{w} = (i, love, hong, kong)$. Let's envision $p(\vec{w})$ as the probability that, if you were to tune in to

the radio at any random moment, the following four words would be "i love Hong Kong." This probability can be evaluated even in the context of a longer sentence, such as "the students of COMP3361 course says, i love Hong Kong more than ever." Often, we analyze $p(\vec{w})$ to compare it with alternative sequences and determine our preference.

Formally, in the context of language modeling, each element W belonging to the event space E represents a potential value of the infinite sequence of words that would be emitted from the radio once it is turned on. The notation $p(\vec{w})$ serves as a shorthand for $p(\text{prefix}(W, |\vec{w}|) = \vec{w})$, where $|\vec{w}|$ represents the length of the sequence $\vec{w}$. Consequently, $p(i, love, hong, kong)$ encapsulates the cumulative probability of all infinite word sequences W that commence with the phrase "i love hong kong...".

Suppose $p(\vec{w}) = w_1 w_2 \ldots w_n$ (a sequence of n words). A trigram language model defines

$$p(\vec{w}) \stackrel{\text{def}}{=} p(w_1) \cdot p(w_2 \mid w_1) \cdot p(w_3 \mid w_1 w_2) \cdot p(w_4 \mid w_2 w_3) \cdots p(w_n \mid w_{n-2} w_{n-1})$$

on the assumption that the sequence was generated in the order $w_1, w_2, w_3, \ldots$ ("from left to right") with each word chosen in a way dependent on the previous two words. (But the first word w_1 is not dependent on anything, since we turned on the radio at an arbitrary moment.)

Q0 (8%)

Expand the above definition of $p(\vec{w})$ using naive estimates of the parameters, such as

$$p(w_4 \mid w_2, w_3) \stackrel{\text{def}}{=} \frac{C(w_2 w_3 w_4)}{C(w_2 w_3)}$$

where $C(w_2 w_3 w_4)$ denotes the count of times the trigram $w_2 w_3 w_4$ was observed in a training corpus.

Note: In practice, we refer to simple parameter estimates like these as "maximum-likelihood estimates" (MLE). The advantage of MLE is that it maximizes the probability, or minimizes perplexity, of the training data. However, they tend to perform poorly on test data.

Q1 (12%)

We discussed the standard Forward Bigram model, which estimates parameters $P_{fwd}(w_i|w_{i-1})$. Now, consider a **Backward Bigram model** that generates text from right to left, estimating parameters $P_{rev}(w_{i-1}|w_i)$.

Instead of counting frequencies from scratch, we want to derive the backward parameters directly from the forward model's parameters.

1. **Derivation:** Use **Bayes' Theorem** to express the backward parameter $P_{rev}(w_{i-1}|w_i)$ in terms of the forward parameter $P_{fwd}(w_i|w_{i-1})$ and the unigram probabilities $P(w_i)$ and $P(w_{i-1})$.

2. **Verification:** Show that your derived expression for $P_{rev}(w_{i-1}|w_i)$ is a valid probability distribution. That is, prove that for a fixed w_i, summing over all possible previous words w_{i-1} equals 1:

$$\sum_{w_{i-1} \in \mathcal{V}} P_{rev}(w_{i-1}|w_i) = 1$$

   (Hint: Use the definition of marginal probability: $P(y) = \sum_x P(y|x)P(x)$.)

Q2 (10%)

Assume we have a vocabulary set $\mathcal{V}$ with size $|\mathcal{V}|$, and we are modeling a sequence of words $\vec{w} = (w_1, w_2, \ldots, w_L)$ of fixed length L.

1. **Full Joint Model:** If we do not make any Markov independence assumptions and attempt to model the full joint probability distribution $P(w_1, w_2, \ldots, w_L)$ directly, how many independent parameters do we need to estimate?
   (Hint: Consider the total number of possible sequences and the constraint that probabilities must sum to 1.)

2. **Trigram Model:** If we instead use a Trigram language model (which assumes a 2nd-order Markov property, i.e., $P(w_i|w_1 \ldots w_{i-1}) \approx P(w_i|w_{i-2}w_{i-1})$), approximately how many parameters do we need to estimate? Please express your answer in terms of $|\mathcal{V}|$.

3. **Pros & Cons Analysis:** Based on your calculations above and your understanding of language modeling, analyze the **advantages and disadvantages** of the N-gram model. Please briefly discuss:
   - One key advantage.
   - One key limitation.
   - One practical challenge when $|\mathcal{V}|$ is large (related to data sparsity).

2 N-gram Language Model Implementation (45%)

In this section, you will build your own language models, evaluate them using Perplexity, and—most importantly—analyze the text they generate.

We provide a dataset with three files containing whitespace-tokenized sentences:

- `train.txt`: Data for training your language models.
- `dev.txt`: Data for tuning hyperparameters (like k in Add-k smoothing).
- `test.txt`: Data for final evaluation.

2.1 Building Vocabulary & Zipf's Law Analysis (15%)

Before handling out-of-vocabulary (OOV) words, it is crucial to understand the distribution of word frequencies in natural language.

1. **Data Analysis:** Count the frequency of all tokens in the `train.txt` file.

2. **Visualization:** Generate a plot of the **Rank-Frequency distribution** of the words.
   - The x-axis should be the rank of the word (1st most common, 2nd most common, etc.).
   - The y-axis should be the frequency of the word.
   - Use a **log-log scale** for both axes.

   Does the plot resemble Zipf's Law?

3. **Discussion:** Based on your plot and data:

- What percentage of total word occurrences in the corpus is covered by the top 10% of the vocabulary?
- Why is the "Long Tail" of low-frequency words problematic for parameter estimation, necessitating the use of an `<UNK>` token?

4. **Preprocessing:** Now, replace tokens occurring less than 3 times in the training data with a special unknown token `<UNK>`. Add start-of-sentence `<s>` and end-of-sentence `</s>` tokens to every sentence. **For evaluation on `dev/test`**, map any token that is not in the training vocabulary $\mathcal{V}$ to `<UNK>`. Report the final vocabulary size $|\mathcal{V}|$.

2.2 N-gram Models & Text Generation (15%)

Build **Unigram**, **Bigram**, and **Trigram** language models using Maximum Likelihood Estimation (MLE).

1. **Perplexity Evaluation:** Report the Perplexity of all three models (Unigram, Bigram, Trigram) on both the `train` and `dev` sets.
2. **Text Generation Implementation:** Implement a function `generate_sentence(model, n=3)` that generates sentences using your trained models.
   - **Sampling Method:** Do **not** use greedy search (i.e., always selecting the word with the highest probability). Instead, use **Weighted Random Sampling** based on the model's predicted probability distribution (e.g., using `numpy.random.choice`).
   - **Reproducibility:** To ensure your results are reproducible for grading, set `random.seed(42)` **once** before generating the 3 sentences for each model.
   - **Constraint:** All generated sentences must start with `<s>` and stop when `</s>` is generated or the length exceeds 20 words.
3. **Output:** Print the 3 sentences generated by each of your Unigram, Bigram, and Trigram models.

2.3 Add-k Smoothing (15%)

Implement **Add-k smoothing** for your Bigram model:

$$P_{Add-k}(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + k}{C(w_{i-1}) + k|\mathcal{V}|}$$

1. **Optimization:** Tune the hyperparameter k on the `dev` set. Try at least four different values (e.g., $k \in \{1, 0.1, 0.01, 0.001\}$) and find the one that minimizes Perplexity.
2. **Evaluation:** Report the Perplexity on the `train` and `dev` sets using your best k.
3. **Generation:** Using your best k, set `random.seed(42)` and generate 3 sentences from this smoothed Bigram model.

2.4 Qualitative Analysis

(This is a discussion question. No new code is needed.)

Based on the sentences generated in Sections 2.2 and 2.3, analyze the quality of the language models.

- **Local vs. Global Coherence:** Compare the sentences generated by the MLE Bigram and Trigram models. Trigram sentences often appear more grammatically correct locally (e.g., phrases of 3-4 words make sense). Do they also exhibit global coherence (i.e., does the whole sentence make logical sense)? Discuss why or why not, referencing the Markov assumption.
- **The Effect of Smoothing:** Compare the sentences generated by the **MLE Bigram model** (Section 2.2) vs. the **Add-k Smoothed Bigram model** (Section 2.3). Did smoothing make the sentences "safer" (containing more common words) or more "random/noisy"? *(Hint: Consider how Add-k affects the probability distribution for unseen or rare context pairs. If k is relatively large effectively, does the distribution become flatter (more uniform)? How does this affect sampling?)*

3 Preposition Prediction with Bidirectional Models (25%)

In this section, you will apply your language models to a practical task: selecting the correct preposition for a given context. You are provided with a development set (`dev.in`) where prepositions are masked with `<PREP>`. Your goal is to predict the correct preposition from a candidate set of 5 options: {`at, in, of, for, on`}.

3.1 Forward Model Baseline (8%)

First, use a **Forward Trigram Model** with add-k smoothing (we use a default $k = 0.1$) to predict the masked prepositions.

1. **Method:** We use an add-k smoothed forward trigram model ($k = 0.1$):

$$P_{\text{fwd}}(p \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}w_{i-1}p) + k}{C(w_{i-2}w_{i-1}) + k|\mathcal{V}|}, \quad k = 0.1.$$

   Since $k > 0$, $P_{\text{fwd}}(p \mid w_{i-2}, w_{i-1}) > 0$ for any $p \in \mathcal{V}$, so the "all-zero" case will not occur. We predict

$$\hat{p} = \arg \max_{p \in \{\texttt{at, in, of, for, on}\}} P_{\text{fwd}}(p \mid w_{i-2}, w_{i-1}).$$

   Note: If multiple candidates obtain the same maximum probability/score, we use `argmax`, which deterministically selects the first one in the fixed candidate order {`at, in, of, for, on`}.

2. **Evaluation:** Report the classification accuracy on `dev.in`.

3. **Submission:** Submit predictions for `test.in` as `test.out1`.

3.2 Bidirectional Inference (10%)

As discussed in **Q1**, context exists in both directions. In a "Fill-in-the-Blank" task, the words *following* the blank are just as important as the words preceding it.

1. **Backward Model:** Construct a **Backward Trigram Model** by reversing each training sentence and training a trigram model on the reversed corpus (using the same vocabulary and preprocessing rules as in Section 2). We apply the same add-k smoothing ($k = 0.1$) to the backward trigram model. This yields probabilities of the form

$$P_{\text{rev}}(w_i \mid w_{i+1}, w_{i+2}).$$

2. **Model Combination:** Implement a **Bidirectional Model** that scores each candidate preposition p by combining probabilities from both the Forward and Backward models:

$$\text{Score}(p) = \log P_{\text{fwd}}(p \mid w_{i-2}, w_{i-1}) \; + \; \log P_{\text{rev}}(p \mid w_{i+1}, w_{i+2}).$$

3. **Experiment:**
   - Predict the prepositions using the combined score above.
   - Report the new accuracy on `dev.in`.
   - Submit predictions for `test.in` as `test.out2`.
   - Compare it with the Forward Baseline. Did adding the right-side context help?

3.3 Error Analysis (7%)

Accuracy numbers don't tell the whole story.

1. **Confusion Matrix:** Generate a 5×5 confusion matrix for your best Bidirectional Model on the `dev` set. The rows should represent the **True** labels, and the columns should represent the **Predicted** labels.

2. **Discussion:**
   - Which pair of prepositions is most frequently confused (e.g., model predicts `in` but label is `at`)?
   - Look at 1–2 examples of these errors in the text. Can you explain why the n-gram model failed?